

Homework 7: Object Oriented Programming

- Due Sunday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 10 at 12am - Jul 24 at 11:59pm

Learning Objectives

- Write functions/methods that raise exceptions when pre-conditions are not met.
- Write classes and methods.
- Use operator overloading.
- Use class attributes.

Things to consider

1. Submit the solutions to all the problems on gradescope. Files to be submitted are: Hw7.py.
2. The output of your program must match the examples exactly for points.
3. This is an individual assignment. Do not use AI or ask your friends for solutions. We will be running similarity checks in gradescope.
4. Run given tests locally before submitting. It will save you a lot of time. Local runs are a lot faster than gradescope submissions.

E-Commerce System

In this assignment, you will build a simplified E-Commerce system consisting of three interconnected classes: Money, Product, and Store. You will need to use operator overloading to handle financial math, class attributes to manage tax rates, and exception handling to prevent invalid operations (like overselling or underpaying).

Write the following 3 classes in Hw7.py.

- **Money**
- **Product**



Store

Money Class

Attributes

The **Money** class objects will have the following attributes.

1. `amount`. This will be a float.
2. `currency`. This will be a String.

The **Money** class will have one class attribute for `exchange_rates`. Class attribute name must be `exchange_rates`.

1. `exchange_rates = {'USD': 1.0, 'EUR': 0.9, 'GBP': 0.8}`

Methods

Methods in the **Money** class are as follows.

- **Constructor (amount, currency) :**

1. `amount` : float
2. `currency` : String

The input should be validated before assignment to the internal attributes.

1. If `amount` is < 0 then raise a **ValueError** with the message "Amount cannot be negative"
2. If `currency` is not in the `exchange_rates` dictionary then raise a **ValueError** with the message "Currency <currency> is not supported" where <currency> is the given currency.

- `convert_to(target_currency)`

Calculate the new amount by converting the current currency to the target currency using the `exchange_rates` dictionary. Return a new Money object with converted amount and currency. Raise a **ValueError** if the target currency is not in the dictionary.

- Overload the mathematical and comparison operators. Convert the `other` object to the currency of `self` before calculation.

- `__add__(other)`
- `__sub__(other)`
 - For subtraction, raise a **ValueError** with the message "Resulting amount cannot be negative" if the result falls below zero.



- `__lt__(other)`
- `__eq__(other)`
- `__str__()`

- The string method should return the rounded amount and currency (e.g., "15.50 USD").

Product

Attributes

The **Product** class will have the following attributes.

1. `name`. This will be a String.
2. `price`. This will be a Money object.
3. `stock`. This will be an integer.

Methods

Methods in the **Product** class are as follows.

1. Constructor (name, price, stock) :

1. `name` : String
2. `price` : Money object
3. `stock` : int

2. `restock(amount)`

1. `amount` : int.

Add to current stock. Raise a **ValueError** if amount is < 0 .

3. `reduce_stock(amount)`

1. `amount` : int.

Subtract from current stock. Raise a **ValueError** with message "Not enough stock available" if amount $>$ current stock.

Store

Attributes

The **Store** class will have the following attributes.

1. `name`. This will be a String.
2. `inventory`. This will be a list of Product objects. Make sure to initialize as an empty list.
3. `revenue`. This will be a Money object starting at 0.0 USD.

The **Store** class will have one **class attribute** for `sales_tax_rate`.



1. `sales_tax_rate` = 0.05

Methods

Methods in the **Store** class are as follows.

1. Constructor (name) :

1. name : String

2. `add_product(product)`

1. product: Product object.

Check if a product with the same name already exists. If it already exists, raise **ValueError** with message "Product already exists". Otherwise, appends to the inventory list.

3. `find_product(product_name)`

1. `product_name` : String.

Loop through inventory and return the Product object if found. Return None if not found.

4. `process_order(product_name, quantity, payment)`

1. `product_name` : String

2. `quantity` : int

3. `payment` : Money object

Look up the product using `find_product`. If None, raise a **ValueError** with the message "Product <name> not found". Calculate the subtotal using a loop. Add tax using the class attribute `sales_tax_rate` to get the grand total. If `payment < grand total`, raise a **ValueError** with message "Payment is less than grand total". Otherwise, call `reduce_stock`, add the grand total to store revenue, and return the change due as a new Money object.

Example (Normal Cases):

```
>> p1 = Product("Laptop", Money(1000, "USD"), 5)
>> my_store = Store("Tech Haven")
>> my_store.add_product(p1)
>> payment = Money(1500, "USD")
>> change = my_store.process_order("Laptop", 1, payment)
>> print(change)
450.00 USD
>> print(p1.stock)
4
```

Example (Exception Cases):

```
>> bad_money = Money(-50, "USD")
ValueError: Amount cannot be negative
>> fake_money = Money(100, "XYZ")
ValueError: Currency XYZ is not supported
>> my_store.process_order("Tablet", 1, payment)
ValueError: Product Tablet not found
>> my_store.process_order("Laptop", 10, payment)
```

```
ValueError: Not enough stock available
>> short_payment = Money(500, "USD")
>> my_store.process_order("Laptop", 1, short_payment)
ValueError: Payment is less than grand total
```

Command Line User Interface (CLI)

At the bottom of your file (inside an `if __name__ == "__main__":` block), create an interactive text-based user interface for managing the store and processing customer orders.

Your CLI must implement the following features:

1. **Setup:** Prompt the user to enter a name for the store, and use that input to instantiate a **Store** object.
2. **Main Menu:** Use an (infinite loop) to repeatedly display a menu with four options: 1. Add Product, 2. Restock Product, 3. Process Sale, 4. Exit.
3. **Add Product:** If the user selects option 1, prompt them for the product name, price amount (float), price currency (String), and initial stock (int). Create the **Money** and **Product** objects, then add it to the store.
4. **Restock Product:** If the user selects option 2, prompt for the product name and the quantity to add. Look up the product using **find_product**, if found, call its **restock** method.
5. **Process Sale:** If the user selects option 3, prompt them for the product name, quantity to buy, payment amount (float), and payment currency (String). Call the **process_order** method and print the change returned.
6. **Exception Handling:** Whenever you instantiate objects or call methods that might raise exceptions (like invalid currency, negative amounts, out of stock, or product not found), wrap the code in a **try...except ValueError as e:** block. Catch the exception, print the error message cleanly to the screen, and allow the infinite loop to continue without crashing the program.

Example Output (Normal & Exception Cases):

```
Enter store name: Tech Haven

--- Tech Haven ---
1. Add Product
2. Restock Product
3. Process Sale
4. Exit
Choice: 1
Enter product name: Laptop
Enter price amount: 1000
Enter price currency: USD
Enter initial stock: 5
Product added successfully!
```

Choice: 1
Enter product name: Laptop
Enter price amount: 1200
Enter price currency: USD
Enter initial stock: 10
Error: Product already exists

Choice: 1
Enter product name: Mouse
Enter price amount: -20
Enter price currency: USD
Enter initial stock: 10
Error: Amount cannot be negative

Choice: 1
Enter product name: Keyboard
Enter price amount: 50
Enter price currency: XYZ
Enter initial stock: 10
Error: Currency XYZ is not supported

Choice: 2
Enter product name: Tablet
Enter restock amount: 5
Error: Product Tablet not found

Choice: 2
Enter product name: Laptop
Enter restock amount: -2
Error: Amount cannot be negative

Choice: 3
Enter product name: Laptop
Enter quantity: 1
Enter payment amount: 500
Enter payment currency: USD
Error: Payment is less than grand total

Choice: 3
Enter product name: Laptop
Enter quantity: 1
Enter payment amount: 1500
Enter payment currency: XYZ
Error: Currency XYZ is not supported

 Choice: 3
Enter product name: Laptop
Enter quantity: 1
Enter payment amount: 1500
Enter payment currency: USD

Success! Your change is 450.00 USD.

Choice: 3

Enter product name: Tablet

Enter quantity: 1

Enter payment amount: 500

Enter payment currency: USD

Error: Product Tablet not found

Choice: 3

Enter product name: Laptop

Enter quantity: 10

Enter payment amount: 15000

Enter payment currency: USD

Error: Not enough stock available

Choice: 4

Exiting program. Final Store Revenue: 1050.00 USD.

Reflection (5% bonus points for submission)

Reflect on the topics covered this week and write short answers for the questions below. Submit as a pdf called Reflection.pdf.

1. What did we learn this week?
2. Rate your understanding of the topics this week. (1. Did not understand at all, ... 5. Fully understand the topic)
3. Are there any questions related to this week's topics? We will address the questions either at the beginning of a lab or through a post.

This tool needs to be loaded in a new browser window

The session for this tool has expired. Please reload the page to access the tool again

